

Can AI Agents Build Game Agents for Real-World Adversarial Games?

Kaisen Yang^{1,*,\dagger}, Qingle Liu^{1,*,\dagger}, Kejin Wang^{1,*}, Yicheng Zhao^{1,*}, Jieming Li^{1,*}, Shenghan Zheng^{1,*}, Ruize Yang^{1,*}, Bojun Yang^{1,*}, Heng Gong¹, Xiang Gao¹, Lanyue Zhang¹, Kaiyu Zhong¹, Zhuo Liu¹, Shaoxuan Li¹, Chengxi Li¹, Yong Yan¹, Weixuan Zhang¹, Tianwei Luo¹, Situ Wang¹, Youjie Zheng¹, Sihan Zhao¹, Shengyuan Wang^{1,2}, Huan-ang Gao¹, Jiazheng Xu¹, Xiaohui Xie¹, Wentao Han¹, Hongning Wang^{1,\ddagger}

¹ Department of Computer Science and Technology, Tsinghua University

² College of AI, Tsinghua University

* Core Contributor; ^{\dagger} Project Lead; ^{\ddagger} Corresponding Author.

AI for adversarial games has progressed from heuristic search to reinforcement learning, yet adapting efficiently to diverse opponents remains challenging. Building on heuristic learning (HL), we formalize Adversarial Heuristic Learning (AHL), a framework in which an AI agent improves executable policies and supporting software through game feedback while its base model’s weights remain fixed. We introduce AAArena, comprising 12 real-world competition games and archived human programs, to evaluate policy development under separate match and full-pool evaluation budgets. Our evaluation of 7 AI models shows that, while Opus5.5 with Claude Code earns 6 gold medals, no evaluated model reaches first place in the other 6 frozen human ladders. Case studies and trajectory analyses reveal persistent difficulties in understanding complex rules, implementing effective strategies, and planning long-term policy improvement. Further experiments show that agents can improve their policies using both on-policy and off-policy replays, with dense feedback yielding better performance than binary feedback.

Contact: yks23@mails.tsinghua.edu.cn, lql24@mails.tsinghua.edu.cn

Project page: <https://aaarena.net>

Code: <https://github.com/THU-CST-SAST/AAArena>



1 Introduction

Games have long served as a testing ground for artificial intelligence, with responding to other agents’ strategies a central challenge throughout this development. Early systems relied on hand-crafted heuristics, evaluation functions, and tree search; reinforcement learning and self-play subsequently advanced experience-driven policy optimization (Campbell et al., 2002; Coulom, 2007; Silver et al., 2018). Across these approaches, opponents form part of each player’s effective environment: different opponents induce distinct, sometimes conflicting strategic modes, and changes in their behaviour shift the learning target. Adapting to these changes under limited interaction while maintaining performance across opponents remains a challenge for continual learning (Hernandez-Leal et al., 2019).

Recent AI agents navigate repositories, execute tests, and revise implementations (Yang et al., 2024). Related computer-use agents extend interaction to graphical interfaces (Xie et al., 2024), while work on agent design and recursive program improvement explores updating the software that supports intelligent behaviour (Hu et al., 2025c; Zelikman et al., 2024).

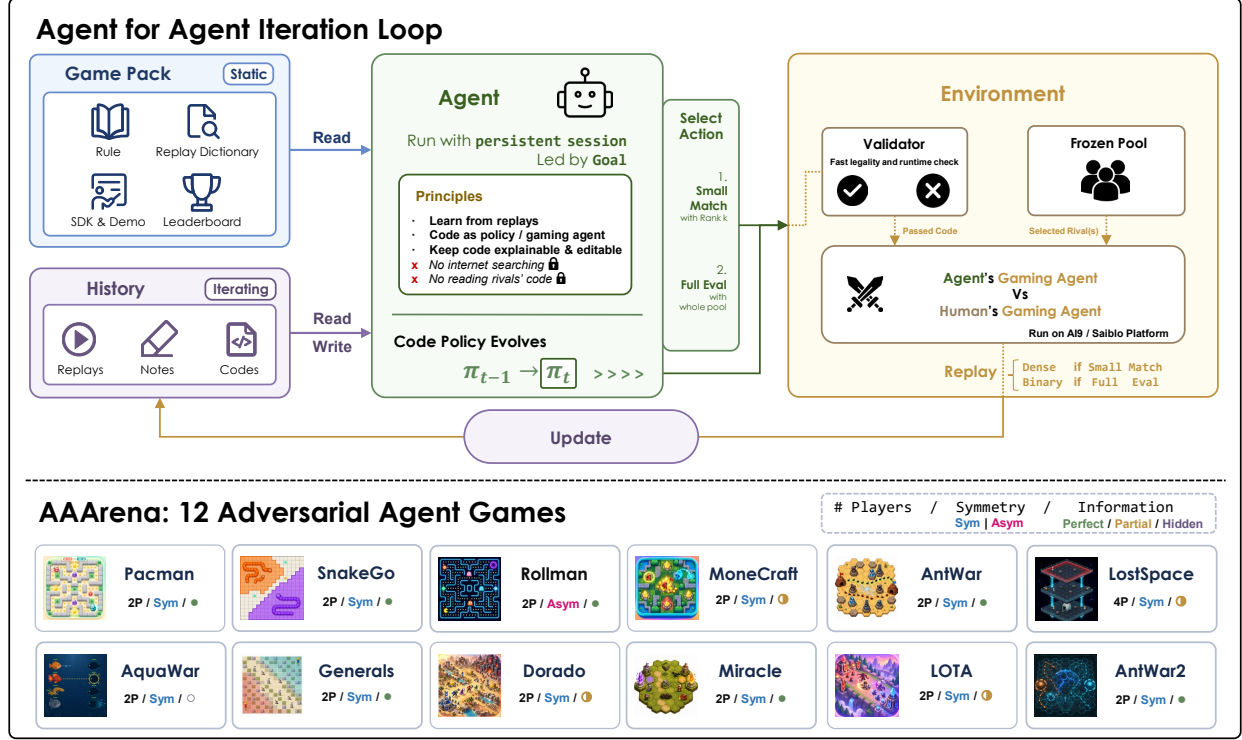


Figure 1 AAArena overview. An AI agent uses game resources, replays, and retained notes to revise an executable game-agent policy. Validated snapshots compete against selected opponents in small matches or the frozen pool in full evaluations. Small matches return dense replays; full-pool feedback reports match outcomes together with Elo and rank. The lower panel presents all 12 games, ordered by rule AST node count, with their player counts, role symmetry, and information structure (Table 1).

Human contestants in game-agent competitions develop and submit programs that play autonomously. An AI agent in this role needs *game understanding* to interpret rules and replays, *strategy implementation* to turn experience into executable policies, and *long-horizon planning* to select opponents and allocate interaction budgets.

Can AI agents take on the role of human contestants, learning from game feedback to build competitive game agents? Weng’s *Learning Beyond Gradients* articulates the related idea of *heuristic learning*: an AI agent reads failures, logs, and replays, then directly improves a program system, accumulating experience in editable software structure (Weng, 2026). Recent work also studies program-policy revision and tournament-based code evolution (Garnier et al., 2026; Wang et al., 2026; Fu et al., 2026; Yang et al., 2026a).

Building on this perspective, we formalize *Adversarial Heuristic Learning* (AHL), the instantiation of HL for competitive game-agent development. The AI agent selects interactions, interprets feedback, and maintains a code policy and its supporting system; the game agent executes that policy against opponents. The base model’s weights remain fixed. The central question is whether sustained program revision can turn limited match experience into competitive strength across a diverse opponent field.

To evaluate this setting, we introduce *Agents for Agents Arena* (AAArena). It brings together 12 original games, runnable backends, and archived human programs from a long-running collegiate competition. Custom rule systems span asymmetric roles, hidden information, resource management, and multi-unit control. Agents obtain detailed replays from selected small matches and Elo and rank from full-pool evaluations. We budget these two feedback channels separately and report the highest officially evaluated score attained within the allowances. Our strongest configuration earns 6 gold medals (rank 1 in the frozen human ladders) across 12 games; no evaluated model tops the remaining 6 ladders.

Our contributions are threefold.

- **We formalize adversarial heuristic learning (AHL)**, a framework for AI agents to develop competitive game agents through feedback-driven revision of executable policies and their supporting systems.
- **We introduce AAArena**, a benchmark combining real competition games, archived human strategies, and a unified interaction interface with a two-budget evaluation protocol.
- **We evaluate multiple AI models on AAArena**, showing policy improvement from on-policy and off-policy replays and gains from dense feedback, alongside persistent gaps in game understanding, strategy implementation, and long-horizon planning in complex games.

2 Adversarial Heuristic Learning

2.1 Preliminaries

Game agent. An adversarial game $\mathcal{G}$ has a shared environment and $n \geq 2$ game agents, with conflicting objectives for at least two agents. Games may have asymmetric roles and partially observable states. Each game agent i executes a *code policy* P_i implementing

$$a_t^i \sim \pi_{P_i}(\cdot \mid h_t^i),$$

where h_t^i is its permitted observation history and a_t^i is its action. The environment processes these actions and produces game outcomes. The submitted program acts autonomously during a match.

AI agent. An AI agent $\mathcal{A}_\theta$ develops the game agent’s policy between matches; θ denotes the base model’s parameters. It reads game resources, source code, and feedback to revise the policy and its supporting software. This separation underlies *Agents for Agents*: we evaluate the AI agent through the competitive performance of the game agent it develops.

Heuristic learning. *Heuristic learning* (HL) incorporates experience through an AI agent’s direct revisions to a software system (Weng, 2026). Feedback may include rewards, tests, logs, or replays. The agent can revise policy logic, state representations, tests, configuration, and memory. The maintained *heuristic system* combines an executable policy with supporting tools, tests, experiment records, and memory that carry experience across revisions.

2.2 Heuristic learning in adversarial games

Definition 1 (Adversarial heuristic learning). *Adversarial heuristic learning (AHL) is an instantiation of HL in adversarial tasks. An AI agent revises the heuristic system from experience and feedback to improve competitive performance, while the base model’s parameters θ remain fixed.*

In our game setting, let P_t be the current policy and C_t its retained development state: supporting tools, tests, experiment records, and memory. A revision is

$$(P_{t+1}, C_{t+1}) = \mathcal{A}_\theta(P_t, C_t, R, F_t; g),$$

where R is the public game material, F_t is the available feedback, and g is the competitive objective. Policy coefficients and development state remain editable.

F_t can include match outcomes, replays, local test results, and permitted evaluation scores. The main experiments use *on-policy* replay feedback from matches involving the learner’s submitted policy. A supplementary experiment tests learning from *off-policy* replays of matches between other policies (Section 4.5). These terms label replay sources; they do not classify AHL as an on-policy or off-policy reinforcement-learning algorithm. Both settings retain development history and full-pool evaluations of the learner’s own policy. Section 3 specifies feedback access and budgets; Appendix B.1 describes the implementation.

Game	Strategic setting	Pool programs	AST nodes	Rules (RA)
Pacman	Competitive maze collection	44	1,127	219
SnakeGo	Snake movement and territory	141	1,538	308
Rollman	Asymmetric maze pursuit	64	1,539	312
MoneCraft	Mining and resource control	112	1,823	347
AntWar	Tower defence and economy	114	2,160	368
LostSpace	Multiplayer survival and escape	111	2,297	426
AquaWar	Hidden-identity tactical combat	170	2,531	439
Generals	Territorial and army control	197	2,683	461
Dorado	Resource competition and army development	323	4,241	616
Miracle	Hex-grid unit tactics	253	4,391	581
LOTA	Hero control and lane combat	200	4,513	684
AntWar2	Ordered tower-defence operations	191	6,368	1,008

Table 1 The 12 games, ordered by increasing rule AST node count. Pool sizes count executable programs, not unique human participants. AST and RA denote rule AST node count and rule count, respectively.

3 AAArena

3.1 Games and opponent pools

The 12 human-authored competition games cover tactical combat, territory, resource management, and survival, each with a verified, frozen opponent pool (Table 1). Agents receive rules, SDKs, replay documentation, a leaderboard, and a permitted reference program. Appendices A and B describe the games and resources.

Rule complexity. Kolmogorov complexity $K(R) = \min_{p: U(p)=R} |p|$ is the length of the shortest program describing R on a fixed universal machine U (Vitányi, 2020). We use $C_{\text{AST}}(G) = |\text{AST}(R_G)|$, the AST node count of the game’s rule representation R_G , as a computable description-size proxy, alongside the rule count RA. These are representation-dependent descriptors, not exact Kolmogorov complexities or direct measures of optimal-play difficulty. The same rule-analysis procedure gives *Go* 501 AST nodes and 125 rules, and *chess* 1,551 AST nodes and 238 rules. These provide familiar reference points: the suite spans 1,127–6,368 AST nodes and 219–1,008 rules, covering a broad range of rule-description sizes.

3.2 Interaction interfaces

Small matches let the AI agent select opponents and inspect detailed replays. **Full-pool evaluations** score a submitted policy against the complete opponent pool, returning Elo, rank, and per-opponent outcomes. Both interfaces evaluate immutable program snapshots through the same game referee, and both return feedback that the agent can use for subsequent revisions.

3.3 Evaluation protocol

Each run receives two budgets: B_s small-match units (one per requested opponent) and B_ℓ full-pool evaluations. Default limits are $B_s = 128$ and $B_\ell = 16$. The agent freely interleaves code revisions and feedback requests within these limits.

The final result is the highest officially evaluated Elo attained during the run, together with its corresponding rank. If $\mathcal{V}$ is the set of officially evaluated snapshots, including the common initialization baseline, then

$$\text{Score}(B_s, B_\ell) = \max_{P \in \mathcal{V}} \hat{r}(P).$$

We retain the policy attaining this score. Elo uses fixed opponent ratings; ranking and budget-accounting details, including the baseline’s separate allowance, are given in Appendix B.

Across games, **Gold medals** count rank-1 finishes in the frozen human ladders (Appendix B.3).

4 Experiments

4.1 Main Results

Table 2 Main results across 12 adversarial games. Each entry gives retained-champion Elo ($\uparrow$) and rank ($\downarrow$) under a 128-small/16-full budget. A gold medal marks rank 1 in the frozen human ladder. The final row reports gold medal counts (Appendix B.3). Bold marks the best value in each row. All models use **max** reasoning effort. Opus5.5 uses Claude Code; all other models use Codex.

Game	Opus5.5		GPT6-sol		GLM-5.3		Kimi K3		DeepSeek V4 Pro		Qwen 3.8		LongCat 2.0	
	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank
Rollman	1010.2	1 🏆	581.2	1 🏆	581.2	1 🏆	636.9	1 🏆	807.7	1 🏆	636.9	1 🏆	534.2	1 🏆
Pacman	2703.9	1 🏆	2367.6	1 🏆	2281.6	1 🏆	2196.3	2	2096.8	4	2236.1	1 🏆	1835.0	10
AntWar	1405.1	5	1311.7	6	1464.2	5	1130.2	9	1167.2	9	1355.0	5	1028.3	11
AquaWar	1552.5	1 🏆	1552.5	1 🏆	1494.1	1 🏆	1494.1	1 🏆	1464.5	1 🏆	1539.7	1 🏆	1116.1	33
Generals	1689.6	10	1528.4	13	1441.2	17	1433.3	17	1340.8	23	1624.9	10	1010.3	58
LostSpace	2220.2	1 🏆	1625.2	7	1980.1	2	1916.1	2	1956.8	2	1797.7	3	1052.9	77
Miracle	1704.8	6	1469.5	20	1436.8	26	1432.8	26	1448.8	24	1500.2	18	1500.2	18
Dorado	2252.4	2	2271.5	2	2210.3	2	2023.4	16	2187.8	3	2035.8	14	1512.4	156
MoneCraft	2387.1	1 🏆	2418.2	1 🏆	2359.2	1 🏆	2288.2	3	2288.2	3	2288.2	3	2069.9	9
LOTA	2224.2	15	2140.3	22	2147.4	22	2133.2	22	2140.3	22	2099.0	25	1585.1	74
SnakeGo	1860.9	1 🏆	1129.6	4	1141.2	4	1165.9	3	1097.0	5	1107.5	5	1153.3	4
AntWar2	2234.4	5	1440.3	34	1977.1	9	1644.2	19	1569.6	25	2183.5	6	1460.0	31
Gold medals $\uparrow$	6		4		4		2		2		3		1	

Setup. Each model receives 128 small-match units and 16 full-pool evaluations per game. Guided by its goal, the agent works in a persistent session, repeatedly reading the rules, writing code, submitting evaluations, and reading replays. The model decides freely when to spend a small-match unit or a full-pool evaluation. A small-match unit plays the candidate policy against one opponent of the agent’s choice and returns dense trajectories; a full-pool evaluation plays it against the entire frozen pool and returns its Elo, its insertion rank, and binary win/loss outcomes for all games. A run stops early once a policy reaches rank 1. We run each setting three times and report the median result. For each trajectory, we report the score of the best policy in that trajectory.

Results. Table 2 and Figures 2–3 summarize the results. Opus5.5 obtains the highest (or tied-highest) Elo on 9 games and earns 6 gold medals. GPT6-sol leads on *Dorado* and *MoneCraft* and ties Opus5.5 on *AquaWar*; GLM-5.3 leads on *AntWar*. Since Opus5.5 runs under Claude Code and the other models under Codex, this comparison mixes model and harness. Among the Codex configurations, GPT6-sol and GLM-5.3 each earn four gold medals, followed by Qwen 3.8 with three. Per-game results also vary: Opus5.5 is the only model above Elo 1200 on *SnakeGo*.

The evaluated configurations collectively earn gold medals on only 6 games. On *LOTA*, *Generals*, and *Miracle*, the best configuration still ranks 15th, 10th, and 6th against human-written agents. Gold medals summarize rank-1 coverage; per-game Elo and rank show the remaining differences. Because analyses require complete interaction logs and a fixed harness, we study GLM-5.3 throughout.

4.2 Case Study and Trajectory Patterns

Dorado: repeated gains through difficult iteration. GLM-5.3’s *Dorado* policy develops from a weak competitor into a near-leading agent, improving from Elo 1760.2 (rank 41) to 2210.3 (rank 2). Early corrections to scoring and defence establish a viable policy; a sequence of economic refinements then drives the main climb, and a conditional **counterattack** delivers the final gain. Progress is uneven, with failed revisions interleaved with advances that reshape the policy’s decision logic. This trajectory illustrates Heuristic Learning: the AI agent turns replay evidence into increasingly effective executable heuristics, making *Dorado* an informative case study (Figure 4).

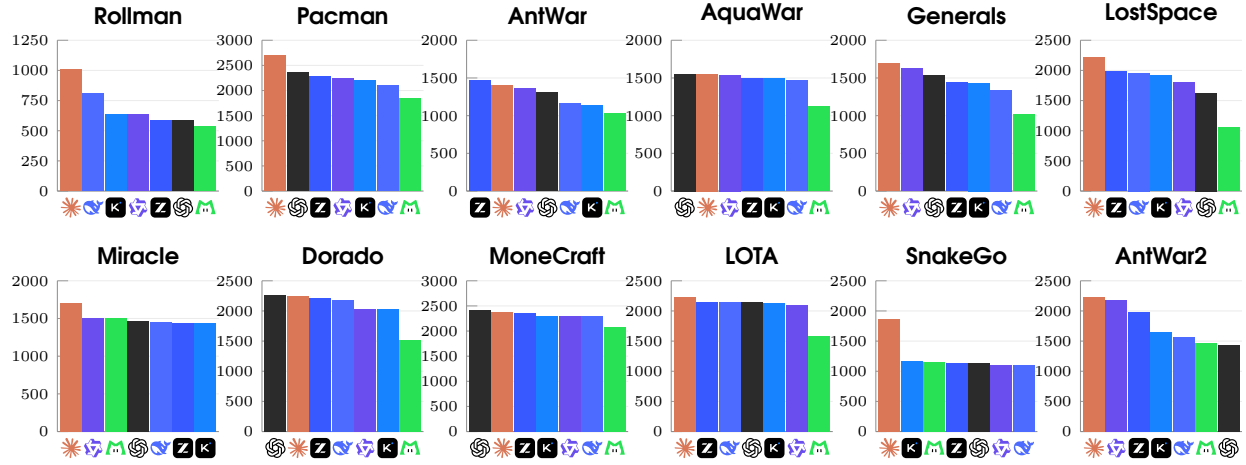


Figure 2 Per-game Elo of the retained champion. Models are sorted by Elo within each game.

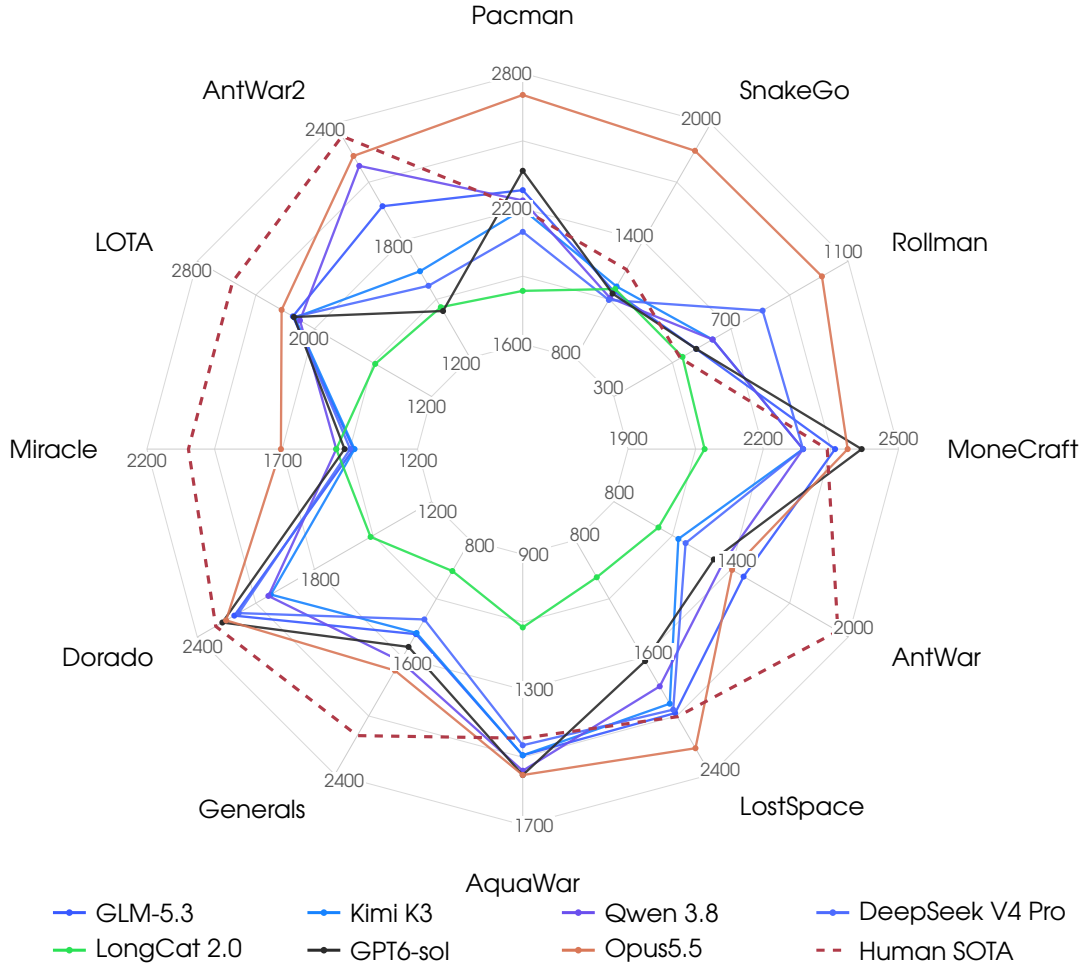


Figure 3 Retained-champion Elo across twelve games. Each spoke reports Elo with a game-specific baseline and evenly spaced increments, labelled at the inner, middle, and outer rings. Higher Elo lies farther outward. The dashed curve denotes human SOTA; model colours match Figure 2.

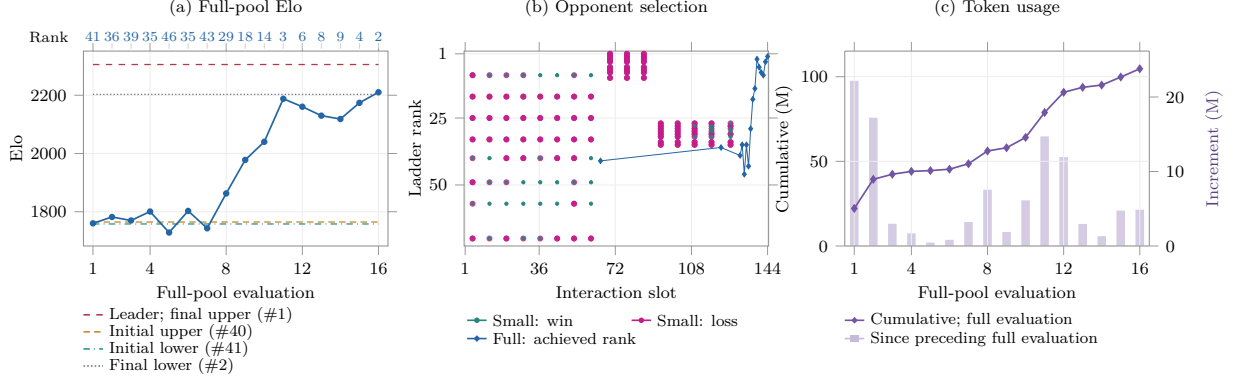


Figure 4 GLM-5.3 on *Dorado*. (a) Full-pool Elo; the aligned row above the curve gives the rank at every evaluation, and reference-line legends give frozen-ladder ranks. (b) Small–full interaction order on 144 equally spaced budget slots (128 small + 16 full). Each opponent test consumes one slot. Points sharing a horizontal coordinate represent multiple opponents in one batch, placed at the midpoint of the batch’s occupied slots. Small-match dots show the selected opponent’s ladder rank, coloured green for wins, magenta for losses, and grey for draws; diamonds show the learner’s achieved rank at full-pool evaluations (rank 1 is at the top). (c) Tokens sampled at the 16 full-pool evaluations (diamonds); bars include all tokens since the preceding full-pool evaluation, with the first bar counting from the run start. The 128 opponent tests comprise 16 batches and 256 role outcomes.

What produces each improvement? Table 3 links each champion improvement to a concise source-code excerpt. The early gains align the policy with **terminal property scoring** and strengthen **base defence**. The main climb from the 8th through the 11th full-pool evaluations corrects the economy: count only living **miners**, discourage duplicate assignments, track **mine opening windows**, and concentrate upgrades on two **core heroes**. The two-hero allocation delivers the largest adjacent gain (+147.7 Elo), reaching rank 3. After several regressions, conditioning a **one-hero counterattack** on sustained nearby aggression reaches rank 2. Progress therefore comes from correcting specific state interpretations and resource decisions; executable changes alone do not ensure better play.

Table 3 *Dorado* champion milestones. Mechanism descriptions with added (+) and deleted (–) C++ diff excerpts. Ellipses omit context.

Full-pool eval.	Elo	Rank	Policy adjustment	Code excerpt
2	1782.0	36	Keep mining ; attack only a vulnerable enemy base .	+232 <code>enemyBase != NULL && enemyBase->hp < 1800</code> && (int)myHeroes.size() >= 4;
4	1800.7	35	Use all living heroes to defend a serious threat.	+245 <code>int count = seriousThreat</code> ? (int)defenders.size() : 2;
6	1802.8	35	Trigger full defence for active threats or recent damage.	+228 <code>const bool seriousThreat = ...</code> <code>baseThreat</code> <code>recentBaseHit</code> ;
8	1862.8	29	Exclude dead miners from occupancy; spread assignments.	+343 <code>if (alive && mineUsable(it->second, round))</code> <code>occ[it->second]++</code> ; -477 ... + 600LL * <code>occ[m]</code> ; +472 ... + 2000LL * <code>occ[m]</code> ;
9	1977.9	18	Penalize duplicate mine assignments more strongly.	-472 ... + 2000LL * <code>occ[m]</code> ; +472 ... + 5000LL * <code>occ[m]</code> ;
10	2040.1	14	Track mine opening windows and prioritize open mines .	+217 <code>s_mineExistUntil[i] =</code> <code>round + exist->val[0] - 1</code> ; + 36 <code>#define MINE_OPEN_OCCUPANCY_PENALTY 5000LL</code>
11	2187.8	3	Concentrate upgrades on a two-hero core ; preserve mining income .	+296 <code>if (s_dutyId < 0</code> && (int)s_leveldCore.size() < 2 ...) +316 <code>if (!coreHammerguard &&</code> <code>type == "hammerguard") priority = 0</code> ;
16	2210.3	2	Allow one-hero counterattacks only after sustained aggression.	+410 <code>s_aggressionRounds = tightAggression</code> ? <code>s_aggressionRounds + 1 : 0</code> ; +412 <code>const bool corePush =</code> <code>activeCoreHeroes >= 2 && aggressionThreat</code> ;

The arena evaluates at least the following abilities of a model.

In-context learning. Effective improvement requires a good understanding of, and insight into, the rules and replays: the model must infer game mechanics, effective play, and causal relationships from them, and distil these into reusable takeaways that survive long-context compaction. The initial Elo probes the model’s zero-shot understanding of the rules, while later scores reflect its long-term understanding of the rules and of replay information. On *Dorado*, the first full-pool evaluation places GLM-5.3 at rank 41 (Elo 1760.2). The model then acquires several key causal insights: only five of the seven **mines** are open at any time, a dead **hero** should not occupy a **mining slot**, and because **terminal property** grows with hero level, **upgrades** should be concentrated on two core heroes rather than spread across all of them. The last insight yields the decisive gain from rank 14 to rank 3 (+147.7 Elo). The case therefore shows in-context learning: the model’s understanding of the game, accumulated over a long session, is what drives its improvement.

Coding ability. The arena also evaluates coding ability, because the agent can act on its understanding of the rules and mechanics only by writing code: in AAArena, the AI agent operates entirely in code space. On *Dorado*, the model writes its own local simulators, including one with rotating mine schedules, smoke tests for specific threat scenarios, and a replay-analysis script; in the policy itself, it implements occupancy-penalized mine assignment, mine-window tracking from visible **existTime**, and a temporal **aggression gate** for counterattacks. Code that compiles is still some distance from code that works. A **four-hero roster** and an unconditional **Sacrifice** both compile and pass local checks, yet drop the policy to rank 46 and 43. After its rank-3 champion, a broad approach-threat defence lowers the rank to 6, and the model calls **restore_champion** to revert before exploring a different change. Beyond verifying an effect, the model must therefore decide whether to roll back to an earlier version and explore anew. The arena thus tests coding ability in a comprehensive way.

Long-horizon planning. When given this freedom, the model usually fixes a batch size (commonly eight) and plays every small match against a whole batch of opponents, so small-match units are consumed quickly. *Dorado*, for example, exhausts all 128 small units before the 3rd full-pool evaluation, leaving 14 full-pool evaluations without fresh small-match replays; later gains therefore rely on stored evidence and offline reasoning. Both **opponent selection** and the **allocation across the small and full budgets** measure long-horizon planning: how to plan a sequence of opponents that yields learning signal over the whole run, and how to coordinate two budgets so that neither is spent unevenly. Current results indicate that this planning ability still has much room for improvement.

Three recurring trajectory patterns. Combining the case analysis above with all 12 GLM-5.3 trajectories, we divide the trajectories into three classes. Success denotes rank-1 early stopping; Progress denotes sustained gains with fluctuations; Stagnation denotes a persistent plateau after initial adaptation. Figures 5–7 show the same three views for every game. Row headings report the retained champion; panel (a) reports each submitted policy’s rank.

- **Success:** *Rollman*, *MoneCraft*, *Pacman*, and *AquaWar* reach rank 1 after 4, 1, 7, and 9 full-pool evaluations, respectively. *Rollman* and *MoneCraft* also leave small-match budget unused (101/128 and 52/128); *Pacman* and *AquaWar* spend all 128 small units but stop before exhausting the full budget.
- **Progress:** *AntWar*, *Dorado*, *LostSpace*, *SnakeGo*, and *AntWar2* obtain results that become stronger overall, rising with fluctuations, although individual submissions can regress: *Dorado* shows several consecutive advances in the middle of the run, *AntWar2* has a sharp intermediate collapse, and *LostSpace* has fewer advances and a long fluctuating tail.
- **Stagnation:** *Generals*, *Miracle*, and *LOTA* enter an extended plateau after their initial adaptation. *Generals* stabilizes around rank 17 after its early climb, *Miracle* repeatedly returns to approximately rank 26–29, and *LOTA*’s best rank remains 22 throughout the full-pool sequence. Additional changes and evaluations mainly revisit similar performance levels.

Overview across the twelve games. The three patterns expose different limits of the same learning loop. Success on *Rollman*, *MoneCraft*, *Pacman*, and *AquaWar* shows that the agent can rapidly connect rules, feedback, and effective code when a useful heuristic is accessible. Progress on *AntWar*, *Dorado*, *LostSpace*, *SnakeGo*, and *AntWar2* shows a slower process of discovering and validating such heuristics; strong final

ranks coexist with costly reversals. Stagnation on *Generals*, *Miracle*, and *LOTA* indicates that continued implementation effort alone does not yield better strategic understanding. Budget planning cuts across these categories: several Progress and Stagnation runs exhaust small-match feedback early.

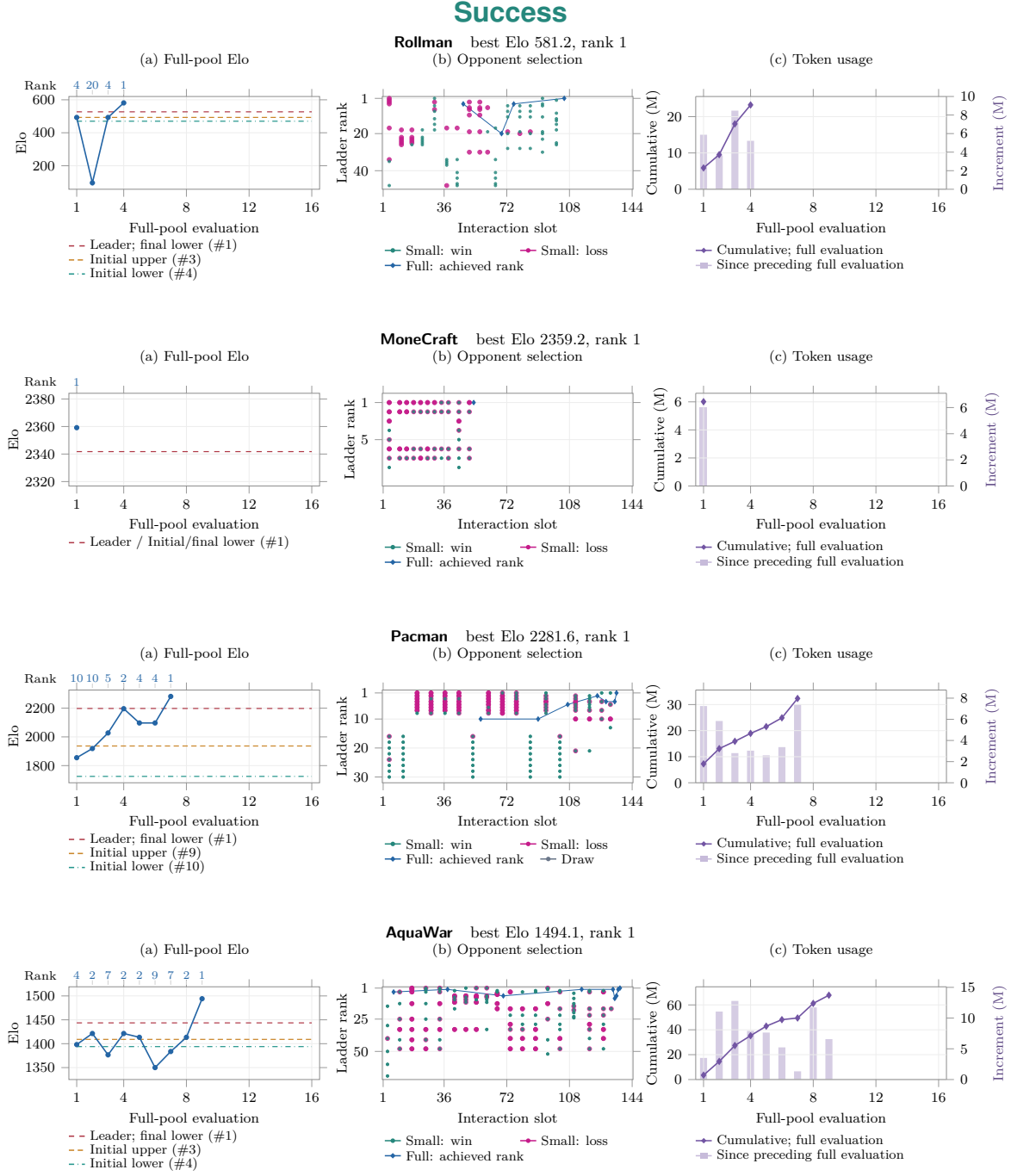


Figure 5 Success. Each game shows (a) full-pool Elo and ranks, (b) small-full interaction order, and (c) token use. Curves end at rank-1 early stopping. Plot conventions follow Figure 4. Points sharing a horizontal coordinate in (b) belong to one batch and consume multiple small-match units.

Progress

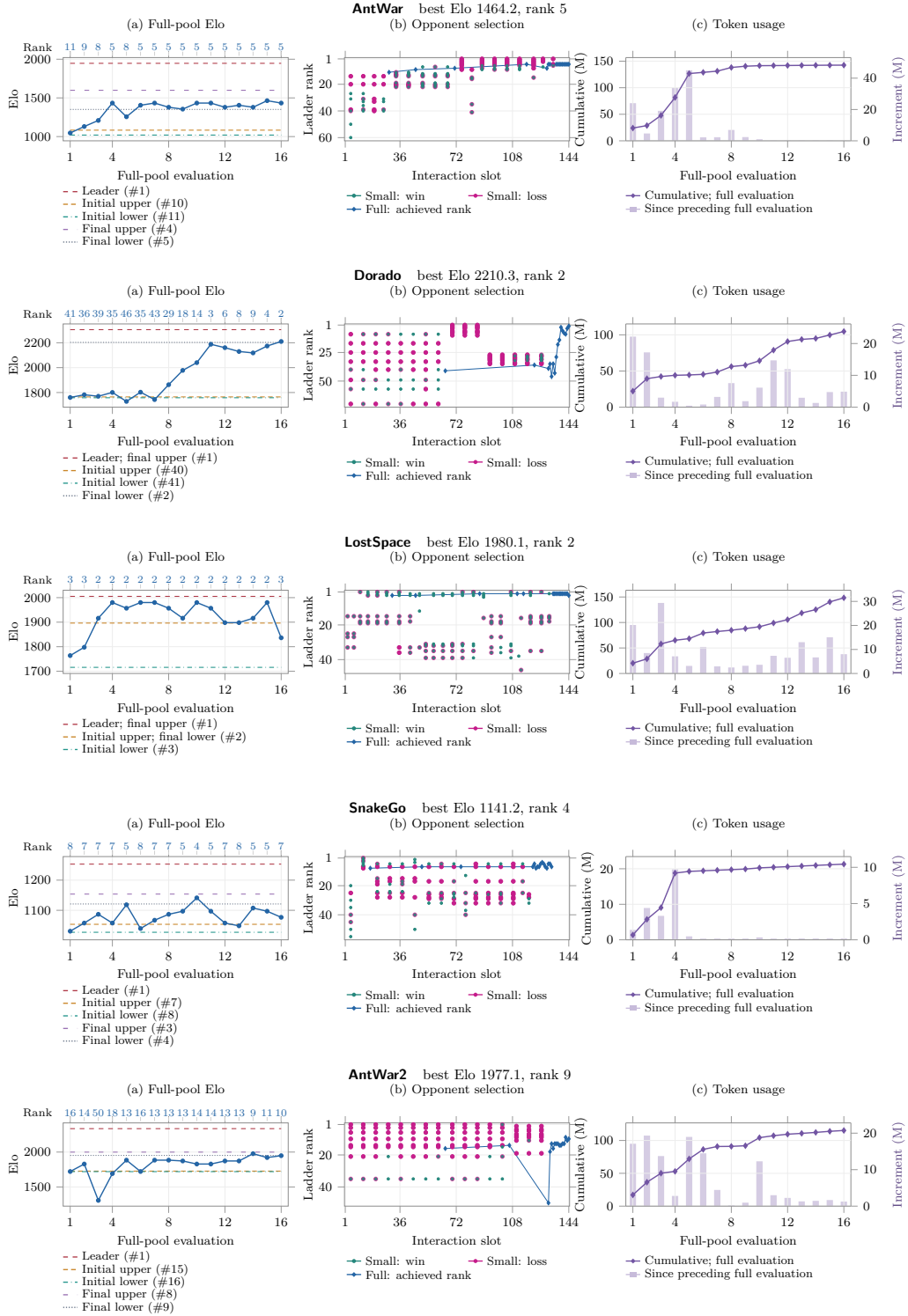


Figure 6 Progress. Games with fluctuating improvement. Each game shows full-pool Elo and ranks, interaction order, and token use, following Figure 4. Points sharing a horizontal coordinate in (b) belong to one batch and consume multiple small-match units.

Stagnation

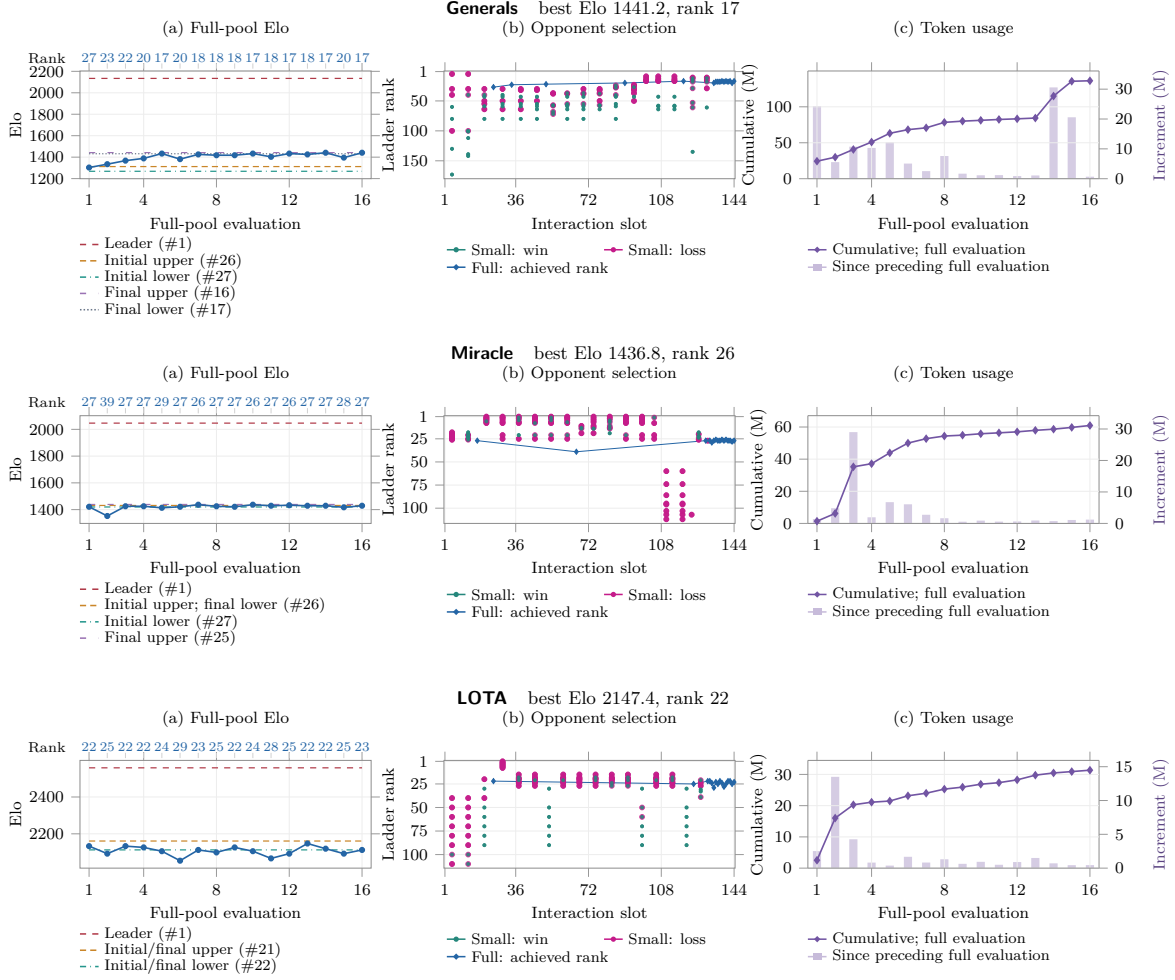


Figure 7 Stagnation. Stagnating games. Each game shows full-pool Elo and ranks, interaction order, and token use, following Figure 4. Points sharing a horizontal coordinate in (b) belong to one batch and consume multiple small-match units.

4.3 Have Low-Performing Games Converged?

For games with weak main-table results, we test whether the low performance is caused by an insufficient budget. We select four games with weak main-stage results and continue GLM-5.3 on them: *Generals*, *Miracle*, *LOTA*, and *AntWar2*. Each continuation inherits the policy workspace and the same evaluation ledger, and adds 256 small-match units and 32 full-pool evaluations. The cumulative budget is therefore 384/48, exactly three times the main-stage 128/16 budget. Table 4 compares the retained champions at the two budget levels. The results show that a threefold budget brings no substantial improvement: the main-table results have largely converged, and the weak performance is not mainly due to the budget.

Table 4 Threefold-budget continuation for GLM-5.3. Elo and rank of the retained champion at each budget. The extension adds 256 small units and 32 full-pool evaluations to the same run. A positive Δ Rank indicates an improved position.

Game	128 / 16		384 / 48		Change	
	Elo	Rank	Elo	Rank	Δ Elo	Δ Rank
Generals	1441.2	17	1441.2	17	+0.0	+0
Miracle	1436.8	26	1486.8	19	+50.0	+7
LOTA	2147.4	22	2147.4	22	+0.0	+0
AntWar2	1977.1	9	2117.0	6	+139.8	+3

Generals and *LOTA* show no improvement. *Miracle* gains 50.0 Elo and moves from rank 26 to 19; *AntWar2* gains 139.8 Elo and moves from rank 9 to 6. Thus tripling the interaction budget produces two unchanged results and two limited residual gains. Figure 8 shows the full 48-evaluation trajectories alongside small–full interaction order and token use. Performance remains flat or fluctuates within a narrow range during most of the extension despite continued expenditure. These trajectories and the limited endpoint gains show that the four games have largely converged under this optimization protocol. This supports the main-table budget as an appropriate evaluation allowance: tripling it yields no improvement or only limited additional gains.

4.4 Ablation Studies

We select one representative game from each trajectory category: *Pacman* (Success), *AntWar* (Progress), and *Miracle* (Stagnation). All ablations use GLM-5.3 with max reasoning effort and the same 128-small/16-full budget. Each axis varies one component while keeping its remaining settings fixed; results are retained-champion Elo, with exact Elo/rank tables in Appendix C.

4.4.1 Does Opponent Selection Affect Performance?

We compare four ways of selecting opponents (model selection, a ladder starting at rank 30, random sampling, and the top four opponents) to examine whether small-match opponent selection affects performance. The main-table configuration uses model selection. Here we additionally implement three selection orders for small matches: the ladder starts at rank 30, and each batch’s opponents rank no lower than those of the previous batch; random sampling draws opponents uniformly from the whole pool; and top-four plays only against the four highest-ranked opponents. In all conditions, the small-match batch size is fixed at four and dense feedback is provided.

Figure 9 shows that model selection and the ladder generally outperform random and top-four selection. The ladder leads on *Pacman* and *AntWar*, while model selection leads on *Miracle*. Restricting feedback to the strongest opponents is consistently weakest. The ladder raises opponent strength as the learner improves, keeping challenge difficulty aligned with current ability. Model selection delegates this matching decision to the agent, which can target weaknesses or test a specific hypothesis. Random sampling may spend feedback on uninformative mismatches, while top-four selection can repeatedly expose failures too severe to suggest an attainable next step. Opponent selection therefore shapes a learning curriculum: the most valuable match provides a discernible direction for improvement at the learner’s current level.

(b) Opponent selection



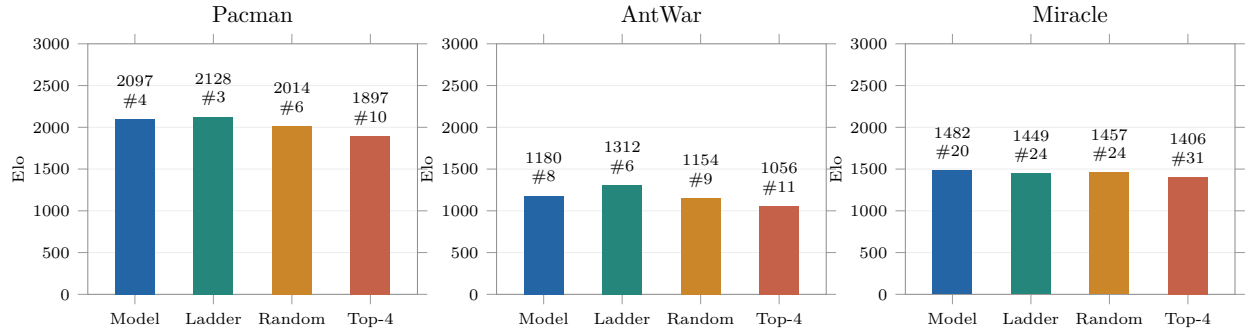


Figure 9 Opponent selection. GLM-5.3 retained-champion Elo with model, ladder, random, and top-four selection; all use dense feedback and batch size 4. Colours distinguish conditions; labels give Elo and rank, and panels share a zero-based Elo axis.

4.4.2 Does Dense Replay Feedback Affect Performance?

Intuitively, dense feedback is a strength of this form of heuristic learning, because it provides small-match replays and their translations, whereas binary feedback provides only win/non-win outcomes. We therefore compare dense and binary feedback directly. Both retain conversation history, model-selected opponents, and full-pool feedback, with batch size chosen by the model. Dense feedback improves Elo on all three games: +526.0 on *Pacman*, +238.1 on *AntWar*, and +67.1 on *Miracle* (Figure 10). Dense feedback is a necessary ingredient for strong HL in this setting. A binary outcome identifies failure but gives little evidence about its cause; replays expose the states, actions, and consequences needed to formulate a code-level correction. This richer information channel is a central advantage of the HL interface over a binary-reward RL interface: it lets the agent reason directly about a failure mechanism and express a targeted revision. The consistent gains across all three trajectory categories show that retaining this diagnostic information is critical to using a limited interaction budget effectively.

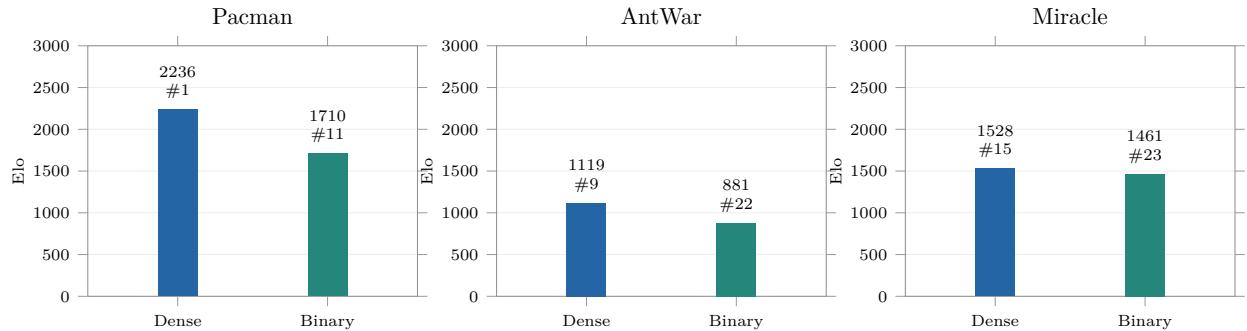


Figure 10 Replay feedback. Dense replays versus binary win/non-win feedback under the same 128/16 budget. Labels give Elo and rank. Dense feedback improves performance in all three representative games.

4.4.3 Does the Number of Opponents per Small Match Matter?

The experiments above fix the number of opponents chosen for each small match; we now examine whether this number matters. We fix the number of opponents per small-match submission to 1, 2, 4, or 8, using model selection and dense feedback. The results show that intermediate batches perform best: size 4 on *Pacman* and *Miracle*, and size 2 on *AntWar* (Figure 11). A single opponent supplies limited evidence per revision, making it difficult to distinguish a general weakness from an opponent-specific interaction. Larger batches expose a candidate policy to more strategies, but a batch of eight can exhaust the small-match allowance in only 16 full-sized requests. Much of that evidence then concerns early policies, leaving fewer chances to test later revisions even when full-pool evaluations remain available. Intermediate batches of 2–4 provide enough variety to compare failure modes while preserving 32–64 opportunities for targeted testing when the small budget is fully used. The appropriate batch size balances evidence breadth, feedback freshness, and the

number of revision cycles; maximizing immediate coverage can undermine long-horizon learning.

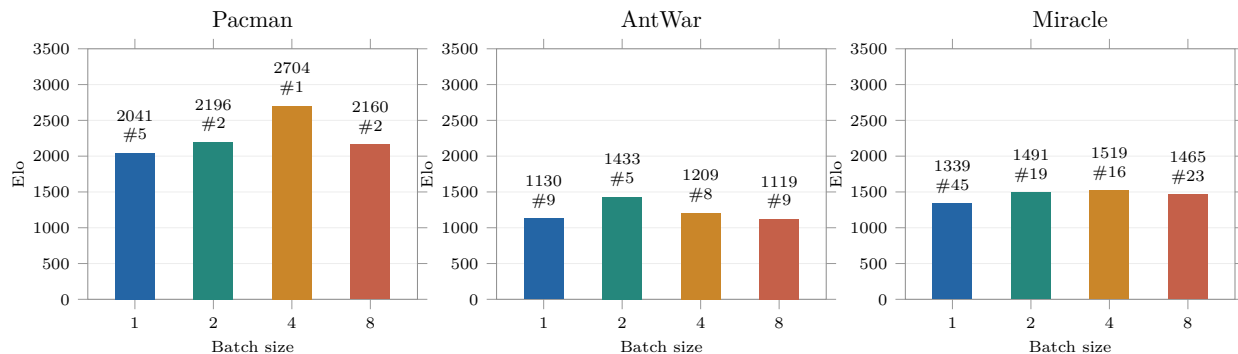


Figure 11 Small-match batch size. Number of opponents per submission, with a fixed total allowance of 128 opponent tests and 16 full-pool evaluations. Labels give Elo and rank. The best size is 4, 2, and 4 for *Pacman*, *AntWar*, and *Miracle*, respectively.

4.5 Can Agents Learn from Off-Policy Replays?

The experiments above learn from on-policy data. We now study whether a model can improve its policy by observing games played by other agents. On *Pacman*, *AntWar*, and *Miracle*, GLM-5.3 receives a catalog of dense replays of matches between frozen-pool opponents and autonomously chooses which opponent pair to inspect. It can request up to 128 replay views and interleave them with up to 16 full-pool evaluations of its own policy, matching the on-policy budget of the main table. As in the main table, it uses max reasoning effort, the same game resources and opponent ladders, and the same 128/16 feedback limits; the off-policy condition changes the source of replay evidence while retaining full-pool performance feedback.

Figure 12 shows improvement in all three games. From the first full-pool evaluation to the retained champion, *Pacman* advances from rank 10 to 1, *AntWar* from 41 to 19, and *Miracle* from 27 to 15. *Pacman* reaches rank 1 after 30 replay views and 14 full-pool evaluations. *AntWar* uses 69 views and all 16 evaluations; *Miracle* uses 128 views and 16 evaluations. The trajectories show that off-policy replays can support policy improvement, while effective allocation of observation and evaluation budgets remains important.

Figure 13 compares the retained champions with the main-table GLM-5.3 runs. The three games illustrate distinct roles for replay evidence.

Pacman is already near the performance ceiling: multiple models reach rank 1 in the main table, and both on-policy and off-policy GLM-5.3 runs reach this target and stop early. Their terminal Elo values (2281.6 and 2404.4) therefore offer little basis for comparing learning quality. Off-policy requires 14 full-pool evaluations, versus 7 on-policy; it attains the same target with more evaluated policy iterations.

On *AntWar*, off-policy reaches Elo 930.7 (rank 19), below on-policy’s 1464.2 (rank 5). Its 69 replay views appear insufficient to recover an effective policy before the 16 full-pool evaluations are exhausted. For this intermediate challenge, on-policy rollouts provide evidence directly tied to the learner’s decisions, making errors easier to locate and its own trajectory easier to correct.

On *Miracle*, off-policy reaches Elo 1528.2 (rank 15), exceeding on-policy’s 1436.8 (rank 26). In this difficult game, weak self-generated rollouts may rarely reveal effective behaviour and thus provide little useful learning signal. Replays of stronger opponents expose strategies that the learner’s own play fails to discover. This result motivates combining external demonstrations with targeted feedback from the learner’s current policy.

On-policy and off-policy learning

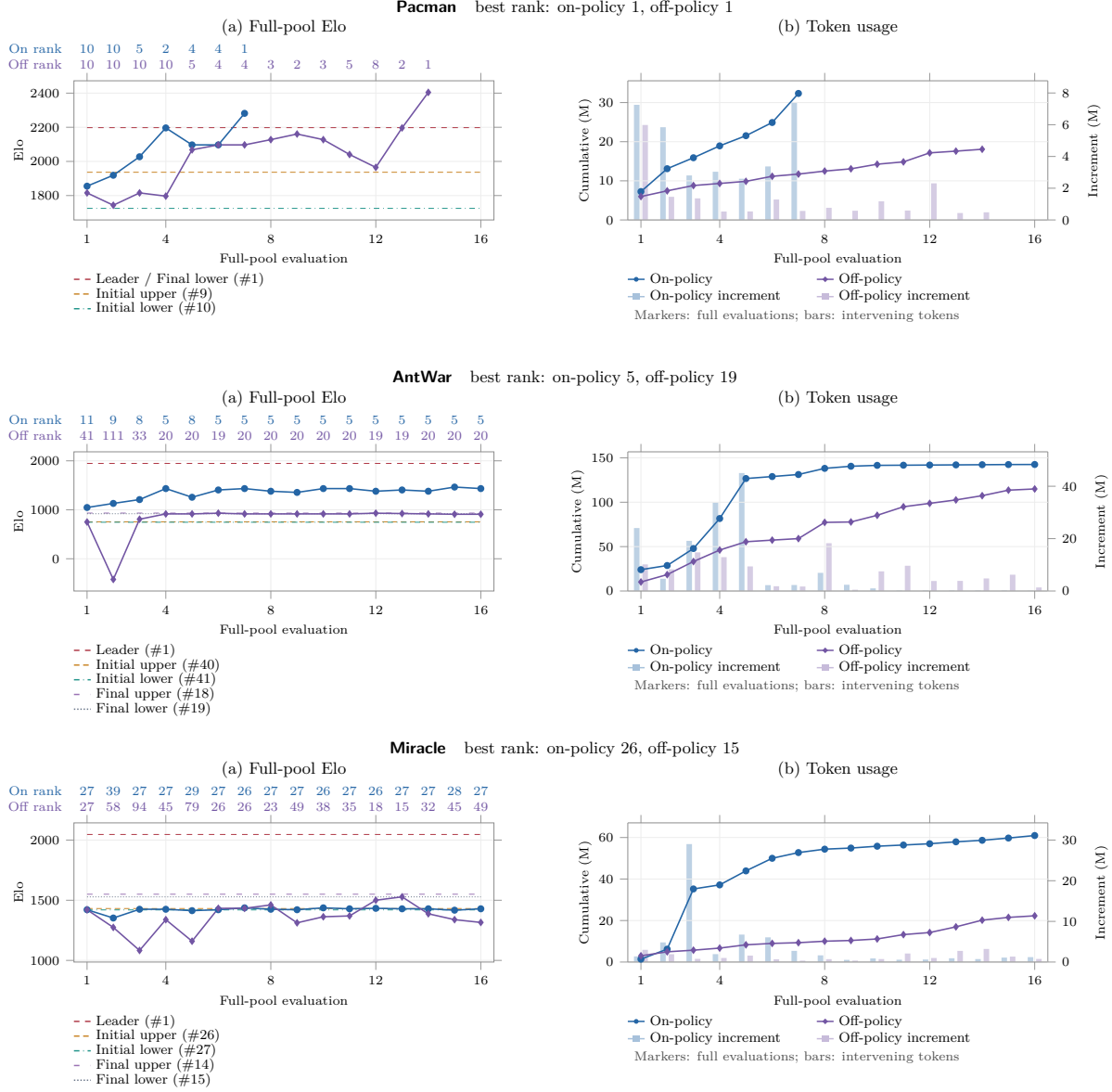


Figure 12 On-policy and off-policy learning trajectories. GLM-5.3 on the same three games; blue circles denote on-policy main-table runs and purple diamonds denote off-policy runs. (a) Full-pool Elo, with aligned rank rows for both conditions. Dashed reference levels and their ladder ranks describe only the off-policy first policy and retained champion. (b) Cumulative tokens at full-pool evaluation submissions (lines) and increments since the preceding full-pool evaluation (paired bars, right axis).

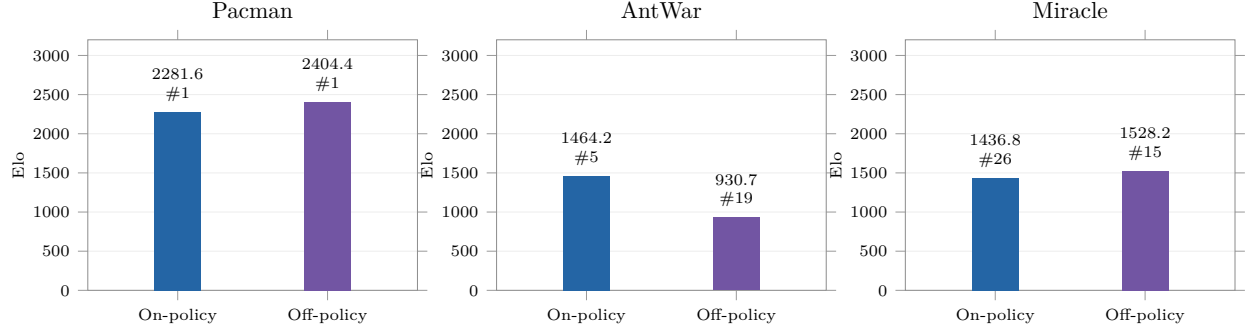


Figure 13 On-policy versus off-policy replay learning. Retained-champion Elo and rank for GLM-5.3 under the same 128/16 budget ceilings. On-policy scores are the main-table results. Labels give Elo and rank; each panel uses the same zero-based Elo scale. Both *Pacman* runs terminate early upon reaching rank 1.

Table 5 Game benchmarks and policy-development protocols. ✓: present in the cited protocol; ×: absent from that protocol; △: partial support or coverage of a subset. Column definitions appear below.

Benchmark / protocol	Custom game suite	Varied gameplay	Human programs	Policy revision	Experiment autonomy	Long-term play
Direct game-playing evaluation						
BALROG (Paglieri et al., 2025)	×	✓	×	×	×	✓
LMGame-Bench (Hu et al., 2025a)	×	✓	×	×	×	✓
GameBench (Costarelli et al., 2024)	×	✓	×	×	×	✓
GameArena (Hu et al., 2025b)	△	△	×	×	×	△
TextArena (Guertler et al., 2025)	△	✓	×	×	×	✓
GAMEBoT (Lin et al., 2025)	×	✓	×	×	×	✓
ZeroSumEval (Khan et al., 2025)	△	✓	×	△	×	△
Kaggle Game Arena (Kaggle, n.d.)	×	✓	×	×	×	✓
Policy development and evaluation						
From Code to Play (Eberhardinger et al., 2025)	×	✓	×	✓	×	✓
Generative code optimization (Kuang et al., 2025)	×	△	×	✓	×	✓
GACL (Game Agent Coding League, n.d.)	×	✓	×	×	×	✓
ProxyWar (Peng et al., 2026)	×	✓	×	△	×	△
Vibe-coding tournament (Danassis and Goel, 2025)	×	×	✓	△	×	✓
CATArena (game track) (Fu et al., 2026)	△	✓	×	✓	△	✓
CodeClash (Yang et al., 2026a)	×	✓	△	✓	△	✓
CC:Ladder (Yang et al., 2026a)	×	✓	✓	✓	△	✓
EvoPolicyGym (Wang et al., 2026)	×	✓	×	✓	✓	✓
AAArena (ours)	✓	✓	✓	✓	✓	✓

Custom game suite: multiple independently designed rule systems; mixed suites and established-game variants receive △. **Varied gameplay:** distinct strategic demands, such as spatial control, resource allocation, and hidden information. **Human programs:** a reusable pool of executable human programs across the evaluated games. **Policy revision:** feedback-driven revision of a persistent policy; repair-only, prompt-only, or one-round refinement receives △. **Experiment autonomy:** the developing agent selects feedback, schedules evaluations, and allocates its budget; autonomy within prescribed rounds receives △. **Long-term play:** multi-step planning or stateful control within a game episode.

5 Related Work

Competitive program development. GACL evaluates generated game agents, while ProxyWar includes code repair (Game Agent Coding League, n.d.; Peng et al., 2026); the vibe-coding tournament compares student and model programs with a self-play refinement condition (Danassis and Goel, 2025). CATArena supplies tournament logs and peer code across revision rounds (Fu et al., 2026). CodeClash studies competitive code evolution, and CC:Ladder uses a prescribed sequence of human opponents (Yang et al., 2026a). EvoPolicyGym

instead budgets episodes in single-agent environments, with hidden validation (Wang et al., 2026). Table 5 compares game diversity, human opponents, and freedom to improve policies through feedback; AAArena combines 12 custom competition games, archived human opponents, and agent-selected matches under separate match and full-pool budgets.

HL and program evolution. Weng’s HL accumulates experience in program policies and supporting software (Weng, 2026). Programmatic reinforcement learning and LLM-guided program search provide related policy representations (Verma et al., 2018; Bastani et al., 2018; Eberhardinger et al., 2025; Kuang et al., 2025). Other work improves controllers, algorithms, and agent software (Garnier et al., 2026; Romera-Paredes et al., 2023; Novikov et al., 2025; Hu et al., 2025c; Zelikman et al., 2024), or revises rewards and training procedures (Ma et al., 2024; Chi et al., 2026). Trajectory distillation extracts reusable skills from replay (Yang et al., 2026b). Our AHL instantiates HL for competitive game-agent development; the base model stays fixed while policies and their development state evolve.

Direct game-playing evaluation. BALROG and LMGame-Bench evaluate models selecting actions in interactive games (Paglieri et al., 2025; Hu et al., 2025a). GameBench, GameArena, TextArena, GAMEBoT, ZeroSumEval, and Kaggle Game Arena extend this evaluation to competitive players (Costarelli et al., 2024; Hu et al., 2025b; Guertler et al., 2025; Lin et al., 2025; Khan et al., 2025; Kaggle, n.d.). These protocols assess the acting player. AAArena assesses the AI agent that develops and maintains an executable player through repeated feedback.

6 Future Work

Extending AAArena. We plan to add games, controlled rule variants, and changing opponent populations to study how game complexity shapes learning. These extensions would broaden evaluation from per-game rankings to adaptation speed, cross-game transfer, and continual learning.

Heuristic and reinforcement learning. Future work should examine how program revision and parameter updates incorporate experience and assign credit for policy improvements. Hybrid learners could combine AI agents’ revisions of policy structure with reinforcement learning of trainable components.

From replay use to active learning. Combining off-policy replays with on-policy interaction could let agents propose strategic hypotheses and select opponents to test them. The next step is to study how agents allocate observations and matches to resolve uncertainty, then consolidate validated strategies into reusable policy modules.

7 Conclusion

We present AAArena, a benchmark for studying how AI agents develop and improve executable game policies through adversarial heuristic learning (AHL). It combines 12 real-world competition games and archived human programs with a unified two-budget evaluation protocol. Our evaluation of 7 AI models shows that Opus5.5 with Claude Code earns 6 gold medals, while no evaluated model tops the remaining 6 human ladders. Further analyses reveal the benefits of dense feedback and learning from both on-policy and off-policy replays, alongside difficulties in game understanding, strategy implementation, and long-horizon planning. We expect AAArena to serve as a foundation for future research on heuristic learning and self-evolving AI agents.

Acknowledgement

We thank the many members of the Student Association of the Department of Computer Science and Technology at Tsinghua University for developing and maintaining the Agent Competition over the years and for supporting this work. We thank Z.ai for sponsoring this work. We also thank all participants in past editions of the Agent Competition.

References

- Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. Verifiable reinforcement learning via policy extraction. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/e6d8545daa42d5ced125a4bf747b3688-Paper.pdf.
- Murray Campbell, A. Joseph Hoane, and Feng-hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1-2):57–83, January 2002. ISSN 0004-3702. doi: 10.1016/s0004-3702(01)00129-1. URL [http://dx.doi.org/10.1016/S0004-3702\(01\)00129-1](http://dx.doi.org/10.1016/S0004-3702(01)00129-1).
- Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement, 2026. URL <https://arxiv.org/abs/2608.20318>.
- Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of llm agents, 2024. URL <https://arxiv.org/abs/2406.06613>.
- Rémi Coulom. *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*, pages 72–83. Springer Berlin Heidelberg, 2007. ISBN 9783540755388. doi: 10.1007/978-3-540-75538-8_7. URL http://dx.doi.org/10.1007/978-3-540-75538-8_7.
- Panayiotis Danassis and Naman Goel. Can vibe coding beat graduate cs students? an llm vs. human coding tournament on market-driven strategic planning, 2025. URL <https://arxiv.org/abs/2511.20613>.
- Manuel Eberhardinger, James Goodman, Alexander Dockhorn, Diego Perez-Liebana, Raluca D. Gaina, Duygu Çakmak, Setareh Maghsudi, and Simon Lucas. From code to play: Benchmarking program search for games using large language models, 2025. URL <https://arxiv.org/abs/2412.04057>.
- Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluating evolutionary capabilities of code agents via iterative tournaments, 2026. URL <https://arxiv.org/abs/2510.26852>.
- Game Agent Coding League. Game Agent Coding League for LLMs. Official benchmark documentation, n.d. URL <https://gameagentcodingleague.com/>. Accessed September 28, 2026; publication date not stated.
- Paul Garnier, Jonathan Viquerat, and Elie Hachem. Heuristic learning for active flow control using coding agents, 2026. URL <https://arxiv.org/abs/2607.11565>.
- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena, 2025. URL <https://arxiv.org/abs/2504.11442>.
- Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6):750–797, October 2019. ISSN 1573-7454. doi: 10.1007/s10458-019-09421-1. URL <http://dx.doi.org/10.1007/s10458-019-09421-1>.
- Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P. Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. lmgame-bench: How good are llms at playing games?, 2025a. URL <https://arxiv.org/abs/2505.15146>.
- Lanxiang Hu, Qiyu Li, Anze Xie, Nan Jiang, Ion Stoica, Haojian Jin, and Hao Zhang. Gamearena: Evaluating llm reasoning through live computer games, 2025b. URL <https://arxiv.org/abs/2412.06394>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025c. URL <https://arxiv.org/abs/2408.08435>.
- Kaggle. Game Arena FAQ. Official benchmark documentation, n.d. URL <https://www.kaggle.com/game-arena-faq>. Accessed September 28, 2026; publication date not stated.
- Haidar Khan, Hisham A. Alyahya, Yazeed Alnumay, M Saiful Bari, and Bülent Yener. Zerosumeval: Scaling llm evaluation with inter-model competition, 2025. URL <https://arxiv.org/abs/2504.12562>.
- Zhiyi Kuang, Ryan Rong, YuCheng Yuan, and Allen Nie. Learning game-playing agents with generative code optimization, 2025. URL <https://arxiv.org/abs/2508.19506>.
- Wenye Lin, Jonathan Roberts, Yunhan Yang, Samuel Albanie, Zongqing Lu, and Kai Han. GAMEBoT: Transparent assessment of LLM reasoning in games. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher

- Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7656–7682, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.378. URL <https://aclanthology.org/2025.acl-long.378/>.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models, 2024. URL <https://arxiv.org/abs/2310.12931>.
- Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. Balrog: Benchmarking agentic llm and vlm reasoning on games, 2025. URL <https://arxiv.org/abs/2411.13543>.
- Wenjun Peng, Xinyu Wang, and Qi Wu. Proxywar: Dynamic assessment of llm code generation in game arenas, 2026. URL <https://arxiv.org/abs/2602.04296>.
- Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475, December 2023. ISSN 1476-4687. doi: 10.1038/s41586-023-06924-6. URL <http://dx.doi.org/10.1038/s41586-023-06924-6>.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419): 1140–1144, December 2018. ISSN 1095-9203. doi: 10.1126/science.aar6404. URL <http://dx.doi.org/10.1126/science.aar6404>.
- Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. Programmatically interpretable reinforcement learning. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5045–5054. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/verma18a.html>.
- Paul M.B. Vitányi. How incomputable is kolmogorov complexity? *Entropy*, 22(4):408, April 2020. ISSN 1099-4300. doi: 10.3390/e22040408. URL <http://dx.doi.org/10.3390/e22040408>.
- Zhilin Wang, Han Song, Runzhe Zhan, Jusen Du, Jiacheng Chen, Tianle Li, Qingyu Yin, Yulun Wu, Zhennan Shen, Tong Zhu, Yanshu Li, Guanjie Chen, Derek F. Wong, Yafu Li, Yu Cheng, and Yang Yang. Evopolicygym: Evaluating autonomous policy evolution in interactive environments, 2026. URL <https://arxiv.org/abs/2607.02440>.
- Jiayi Weng. Learning beyond gradients. <https://trinkle23897.github.io/learning-beyond-gradients/>, May 2026. Blog post.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Muhtasham Oblokulov, Aryan Siddiqui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering, 2026a. URL <https://arxiv.org/abs/2511.00839>.
- Kaisen Yang, Zheng Jiang, Yuzhao Peng, Houde Qian, Boshi Zhang, Youjie Zheng, Shijin Hong, Qingle Liu, Ruoyu Han, Bohan Lyu, Bingxiang He, Eren Cai, Calvin Xiao, and Qinhuai Na. Scalable behaviour cloning on browser using via skill distillation, 2026b. URL <https://arxiv.org/abs/2606.32014>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation, 2024. URL <https://arxiv.org/abs/2310.02304>.

A The evaluated games

The suite shares an evaluation interface, but its policies solve different decision problems. The descriptions below refer to the public rule and SDK bundles used in the experiments; pool sizes are listed in Table 1.

Pacman. *Pacman* is a separate, symmetric competitive maze game. Players collect ordinary and **high-value beans**, avoid **monsters**, and place **mines** that affect movement choices. Collection, collision avoidance, and disruption of the opponent jointly determine the score. Its rules and opponent pool are distinct from *Rollman*’s asymmetric contest.

SnakeGo. *SnakeGo* is a territory game in which **snakes** move, collect items, claim **cells**, and can split or use special actions. Local collision avoidance interacts with longer-horizon territory growth, making a safe immediate move potentially costly at settlement.

Rollman. *Rollman* is an asymmetric maze game. One side collects **beans** and uses temporary skills; the other coordinates **ghosts** to intercept it. The roles differ in both action space and objective, making movement planning and coordinated pursuit distinct policy problems.

MoneCraft. *MoneCraft* is a mining and resource-control game on a symmetric map. Players capture **mines** that generate **currency** and can teleport, place **mines**, or purchase protection. The policy trades movement and disruption against sustained resource production; end-game **currency** and **mine ownership** determine the result under the referee’s rules.

AntWar. *AntWar* combines tower defence with economic management. Players build and upgrade **towers**, improve their **bases**, and time **super-weapons**, while **ants** follow environment-driven paths. The policy must balance present defence with spending that changes future pressure.

LostSpace. *LostSpace* is a multiplayer survival-and-escape game on a layered map that contracts over time. Players collect **supplies** and **keys**, use **tools**, fight or evade rivals, and seek escape. Ranked outcomes create trade-offs between immediate combat, survival, and the timing of an escape attempt.

AquaWar. *AquaWar* is a hidden-identity tactical game. Players select **fish**, make assertions about hidden opponent identities, and choose attacks or type-specific skills. Roster design, inference, and information-sensitive timing all affect the outcome.

Generals. *Generals* combines territorial expansion, army movement, neutral assets, production, technology, and tactical abilities. An effective policy coordinates economic development with the timing and direction of attacks rather than optimizing either in isolation.

Dorado. *Dorado* combines resource competition with **hero** and army development. Players choose **heroes**, produce and command **units**, mine neutral resources, and invest in **upgrades**. Movement, combat, harvesting, and abilities must be coordinated to threaten opposing **bases** while maintaining the resources needed to continue production.

Miracle. *Miracle* is a hex-grid strategy game with heterogeneous **units** and an initial drafting stage. A player selects **creatures** and an **artifact**, then summons, moves, attacks, and uses abilities to destroy the opponent’s **miracle**. Resource limits and unit interactions make tactical coordination central to the policy.

LOTA. *LOTA* is a lane-based combat game. Each player chooses a **hero** and controls movement, attacks, abilities, skill development, and **revival** while **units** advance along fixed **lanes**. Victory depends on destroying the opposing **base** or the referee’s settlement criteria at the turn limit. Effective control couples immediate fights with sustained pressure on defensive structures.

AntWar2. *AntWar2* extends tower-defence strategy with **ordered operation lists** whose accepted prefix changes subsequent decisions. The agent must coordinate **tower placement**, economic development, and weapon timing while respecting sequence-dependent legality.

B Arena resources and evaluation details

Public resources and opponents. Each opponent is a versioned executable program, and the opponent pool, opponent ratings, and game resources are frozen for all runs. Because some entrants submitted related versions, pool sizes count programs rather than independent human participants. The agent’s workspace contains the rules, the SDK, replay-format documentation, the leaderboard, and one permitted reference program (the rank-40 entry). Other opponents’ source code and the evaluator internals are not visible.

Budget accounting. A small-match request may contain up to 8 opponents and is charged one unit per opponent; a full-pool evaluation is charged one large unit. The budgets limit only feedback: source edits, model calls, tokens, and wall-clock time are not restricted. The initial evaluation of the starter policy is not charged and does not count toward the full-pool sequence.

B.1 AI agent workflow

Each model–game run is a single persistent agent session with a writable workspace: Opus5.5 runs in Claude Code and all other models run in Codex. The agent is given the goal of improving competitive performance and decides on its own when to read the rules, edit its program, request small matches against opponents of its choice, or submit a snapshot for full-pool evaluation. Small matches return full replays; full-pool evaluations return Elo, rank, and per-opponent outcomes. The session persists across turns, and the harness compacts the context when it grows too long, so the agent must keep its own notes to carry knowledge across compactions. The reported result is the highest officially evaluated Elo attained within the run, including the common initialization baseline, together with the corresponding frozen-ladder rank. The controller retains the snapshot attaining this score.

B.2 Elo and rank

For each full-pool evaluation of a policy snapshot P , we estimate its Elo $\hat{r}(P)$ from that evaluation’s match outcomes using a fixed-pool Bradley–Terry model. Let H be the frozen opponent pool, r_j the fixed rating of opponent j , and $s_m \in \{0, \frac{1}{2}, 1\}$ the candidate’s score in match m against opponent $j(m)$ (loss, draw, win). The expected score at rating r is

$$p(r, r_j) = \frac{1}{1 + 10^{(r_j - r)/400}},$$

and the estimate $\hat{r}$ solves

$$\sum_m p(\hat{r}, r_{j(m)}) + p(\hat{r}, \bar{r}_H) = \sum_m s_m + \frac{1}{2}, \quad \bar{r}_H = \frac{1}{|H|} \sum_{j \in H} r_j.$$

The extra term is one virtual draw at the mean rating of the complete frozen pool, which keeps $\hat{r}$ finite for perfect win or loss records. Opponent ratings remain fixed across snapshots and runs. The candidate’s rank is $1 + |\{j \in H : r_j > \hat{r}\}|$. We report each game’s native Elo and rank, and summarize rank-1 coverage with gold medals.

B.3 Gold medals

Gold medals count rank-1 finishes against the frozen human ladders. For configuration m and game g , let $q_{m,g}$ denote its reported rank. Across $G = 12$ games, we compute

$$\text{Gold}(m) = \sum_{g=1}^G \mathbf{1}\{q_{m,g} = 1\}.$$

A gold medal denotes first place under this rating protocol; it does not establish a direct head-to-head win against the human leader or optimal play.

The pool ratings and rank-1 thresholds remain fixed when we evaluate additional models. Gold medals summarize performance against the archived pools. Full-pool evaluations provide feedback during optimization, so the reported results do not measure generalization to unseen opponents. The committed summary records pin the rating inputs and calculations; they do not independently validate the original human-rating calibration.

B.4 Rule complexity: AST and RA

To compare rule complexity across games, we rewrite each game’s rules in a small formal pseudocode language and measure the size of the resulting description. The language has a fixed, domain-neutral vocabulary: top-level declarations (`constant`, `enum`, `entity`, `action`, `observation`, `setup`, `rule`, `terminal`), fifteen statement types (e.g. `let`, `require`, `when`, `for`, `update`, `create`, `delete`, `reveal`, `return`), and a restricted expression syntax for arithmetic, comparisons, and function calls. Generic mathematical concepts such as set membership, distance, and probability are primitives, whereas every game-specific operation (for example, splitting a snake or casting a fish skill) must be defined explicitly as a rule. Each description covers the player and information model, state, initialization and randomness, legal actions, transitions, hidden information, and terminal conditions; communication protocols, rendering, logging, and platform-only timeouts are excluded.

AST. The primary measure is the node count of the description’s canonical abstract syntax tree,

$$C_{\text{AST}}(G) = N_{\text{structure}}(d_G) + N_{\text{expression}}(d_G),$$

where d_G is the formal description of game G , $N_{\text{structure}}$ counts declaration, block, and statement nodes, and $N_{\text{expression}}$ counts expression nodes (calls, names, attributes, literals, collections, subscripts, comparisons, and operators). Parser-only wrapper and context nodes are not counted. A named rule is counted once where it is defined and once at each call site.

RA. The secondary measure, rule atoms (RA), counts independently changeable rule propositions. Each declaration member or rule statement expresses exactly one such proposition; two effects written in one sentence still count as two. RA is the size of the disjoint union of seven proposition classes: state, action, observation, setup, condition, transition, and outcome.

Both measures are invariant to identifier renaming, comments, and formatting. They depend on the chosen language and measure the size of the rule description, not the state-space size or the difficulty of optimal play.

C Detailed ablation results

Tables 6–8 list the Elo and rank behind Figures 9–11.

Game	Model	Ladder	Random	Top-4
Pacman	2096.8 / 4	2127.5 / 3	2014.4 / 6	1897.0 / 10
AntWar	1180.5 / 8	1311.7 / 6	1154.4 / 9	1056.2 / 11
Miracle	1482.4 / 20	1448.8 / 24	1457.0 / 24	1406.0 / 31

Table 6 Opponent selection ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	Dense	Binary
Pacman	2236.1 / 1	1710.1 / 11
AntWar	1118.7 / 9	880.6 / 22
Miracle	1528.2 / 15	1461.1 / 23

Table 7 Replay feedback ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	1	2	4	8
Pacman	2040.6 / 5	2196.3 / 2	2703.9 / 1	2160.4 / 2
AntWar	1130.2 / 9	1433.4 / 5	1208.7 / 8	1118.7 / 9
Miracle	1338.7 / 45	1491.2 / 19	1518.7 / 16	1465.3 / 23

Table 8 Small-match batch size ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	Opus5.5	GPT6-sol	GLM-5.3	Kimi K3	DeepSeek V4 Pro	Qwen 3.8	LongCat 2.0
Rollman	48.75	25.28	23.82	5.50	6.11	14.68	18.35
Pacman	22.16	16.14	33.14	40.17	91.95	163.12	17.52
AntWar	74.19	16.08	142.94	54.59	102.80	284.29	37.56
AquaWar	76.95	26.37	68.68	1.49	60.11	13.92	20.65
Generals	75.44	19.66	137.23	88.88	172.70	60.95	22.30
LostSpace	65.37	21.58	149.86	72.03	141.51	190.16	100.95
Miracle	89.29	19.15	61.99	34.78	47.89	107.55	49.05
Dorado	44.58	23.02	105.63	60.72	67.01	253.02	10.14
MoneCraft	56.16	22.10	6.75	29.97	67.54	48.72	31.18
LOTA	76.41	17.36	32.55	30.74	34.04	112.36	21.92
SnakeGo	48.53	23.53	22.49	59.75	29.79	78.77	27.35
AntWar2	58.78	30.07	115.70	71.08	44.08	126.53	67.20
Total	736.61	260.34	900.78	549.70	865.53	1454.07	424.17

Table 9 Token use of the main-table runs (millions, input plus output; cached input is counted once as part of input). All runs share the 128/16 interaction budget.

D Token consumption

The main text summarizes retained-champion performance. Under the same small-match and full-pool limits, the models consume different numbers of tokens. Table 9 reports the token use of every main-table run.

Token use varies by more than a factor of five across models under identical interaction budgets. GPT6-sol uses the fewest tokens: 260M in total and between 16M and 30M on every game. Qwen 3.8 uses 1,454M, the most of any model. Qwen 3.8 consumes 1.6 times as many tokens as GLM-5.3 but earns three gold medals versus four. GPT6-sol earns four gold medals, matching GLM-5.3, with less than a third of GLM-5.3’s tokens. Opus5.5 with Claude Code, the leading configuration, consumes 737M. Within a model, consumption also varies widely across games; for example, Kimi K3 uses 1.5M tokens on *AquaWar* but 89M on *Generals*. These differences reflect how much each model reads, reasons, and revises before spending its feedback budget.

E Algorithm families in the champion policies

Because the agent acts entirely in code space, the final policies also show which algorithms each model chooses to write. We classify the code of all 84 retained champions into six families:

- **Rule-based control:** hand-written if-else priorities, state machines, and fixed openings or scripts.
- **Heuristic scoring:** hand-weighted features that score candidate actions or targets, followed by a greedy choice.
- **Graph search:** BFS, DFS, Dijkstra, A*, distance maps, flood fill, and territory partitions.
- **Lookahead search:** search over a forward model, including simulated rollouts, beam search, minimax, expectimax, and MCTS.
- **Combinatorial optimization:** assignment and matching, dynamic programming, and knapsack-style allocation.
- **Inference and randomization:** belief tracking over hidden information, opponent modelling, and mixed or randomized choices.

We count the source files reachable from each policy’s entry point, excluding unmodified SDK files, vendored libraries, and leftover version, backup, debug, and test copies. Each function is assigned one family from its structure and identifiers; functions longer than 40 lines are split into 20-line windows that are classified separately. I/O, parsing, and protocol code are counted as infrastructure and excluded from the denominator. A family’s share is its fraction of the remaining non-comment lines. Table 10 reports the shares for every model–game pair, with equally weighted model and game means.

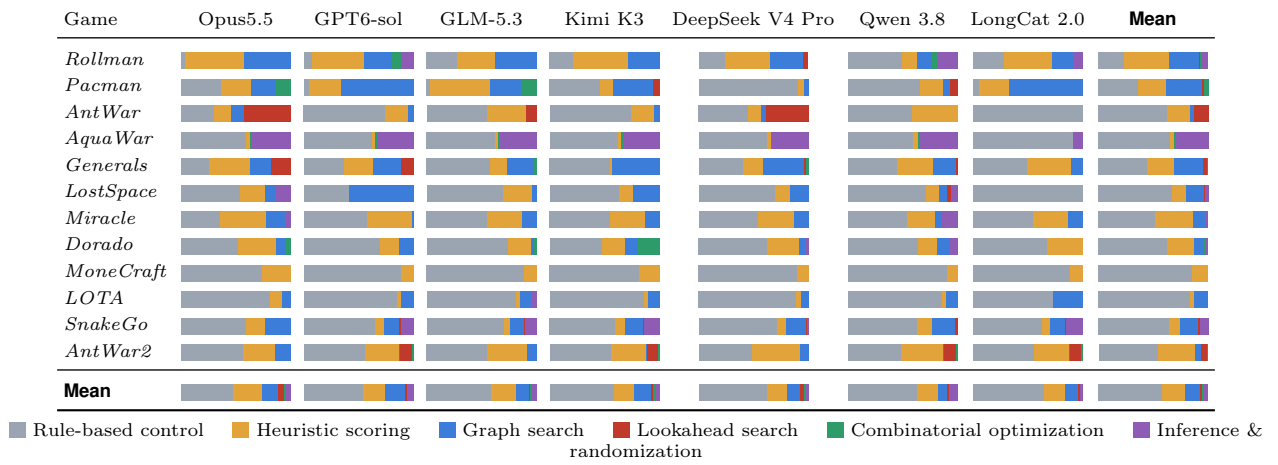


Table 10 Algorithm families in the champion policies. Each bar splits the decision code of one retained champion by algorithm family, weighted by non-comment lines of code. Infrastructure (I/O, parsing, protocol) is excluded. Row and column means weight each policy equally.

Hand-written heuristics dominate. Rule-based control and heuristic scoring together make up 79% of the decision code, graph search adds 13%, and the three remaining families together account for 8%. Lookahead search is rare: it exceeds 10% of a policy in only 7 of the 84 champions, all on *AntWar*, *AntWar2*, and *Generals*, and no final policy runs MCTS, even though the *AntWar2* SDK ships an MCTS training script. The agents thus improve their policies mainly by refining rules and weights, not by adopting stronger search.

The game shapes the composition more than the model does. Maze games such as *Pacman* and *Rollman* are built around graph search (33% and 28%), *AquaWar* is the only game in which inference over hidden identities is substantial (30%), and *MoneCraft* and *LOTA* are almost entirely rules. Across models, the mean shares differ much less. Opus5.5 writes the least rule-based code (47%) and the most heuristic scoring (27%) and lookahead search (5%), and GPT6-sol uses the most graph search (18%); the other models range from 58% to 65% rules. Within a game, a smaller share of rule-based code tends to accompany a higher Elo (mean Spearman correlation 0.35 over the 12 games), but the correlation varies widely across games.

The classification is automatic and approximate. Identifiers and comments influence it, and a function that mixes several mechanisms is assigned to its dominant one. Where several models kept the same starter code, as on *AquaWar* and *LOTA*, their shares are similar.